

Trabajo Práctico Grupal Stacks Enterprise de Datos

Gestión y Arquitectura de Datos, Universidad de San Andrés

1. Introducción

A lo largo del curso vieron distintos paradigmas para gestionar datos: bases relacionales, normalización, SQL, y las cuatro grandes familias de NoSQL (documental, clave-valor, columnar y grafos). También discutimos el teorema CAP y los trade-offs que aparecen cuando un sistema crece.

En el mundo real, las empresas no usan una sola base de datos: combinan varias tecnologías en lo que se conoce como un **stack de datos**. Un stack es un conjunto de herramientas integradas que cubren todo el ciclo de vida del dato: ingesta, almacenamiento, procesamiento, transformación y consumo final (dashboards, reportes, aplicaciones).

El objetivo de este trabajo es que investiguen un stack enterprise real, entiendan cómo se integran sus piezas, y se lo expliquen al resto de la clase. La presentación se realizará en la **Clase 10**, y servirá como punto de partida para la discusión sobre arquitecturas analíticas.

2. Modalidad

- Se dividirán en **grupos de 2 a 4 personas**.
- Cada grupo recibe un stack **por sorteo** al final de la Clase 9. No puede haber dos grupos con el mismo stack.
- La entrega consiste en una **presentación oral grupal** de 12 a 15 minutos, seguida de 5 minutos de preguntas.
- Las slides son obligatorias. Todos los integrantes deben hablar durante la exposición.
- La presentación se realiza en la **Clase 10**.

3. Stacks Disponibles

3.1. Stack 1: Google Cloud Platform (GCP)

El stack de datos de Google. Tiene sentido que sea uno de los más fuertes del mercado porque Google viene manejando volúmenes absurdos de datos desde hace décadas (búsqueda, YouTube, Gmail) y mucho de lo que después se volvió estándar en la industria salió de papers de Google: MapReduce, BigTable, Dremel.

El componente central es BigQuery, un data warehouse serverless. Serverless quiere decir que el usuario no administra ni servidores ni clusters: tira queries y Google se encarga del resto. Se cobra por la cantidad de datos que escanea cada query. Lo eligen mucho las empresas que no quieren pelearse con infraestructura.

Componentes principales: **BigQuery**, **Cloud Storage**, **Dataflow**, **Pub/Sub**, **Dataproc** y **Looker**.

3.2. Stack 2: Amazon Web Services (AWS)

AWS fue el primero en el negocio del cloud moderno (lanzó S3 en 2006) y hoy sigue siendo el que más mercado tiene. Su stack de datos es enorme: tiene un servicio para prácticamente cualquier problema, lo que también significa que hay que conocer muchas piezas para armar algo razonable.

S3 es la base de casi todo el stack analítico de AWS y probablemente sea el sistema de almacenamiento más usado del mundo. Comparado con GCP, AWS es más modular: te da las piezas, vos las integrás. Más flexibilidad, más complejidad.

Componentes principales: **Amazon S3**, **Redshift**, **AWS Glue**, **Kinesis**, **Athena**, **EMR** y **QuickSight**.

3.3. Stack 3: Microsoft Azure

El stack de datos de Microsoft. Donde más pega es en empresas que ya viven en el mundo Microsoft: Office 365, Active Directory, SQL Server, Windows Server. Si ya usás Power BI todos los días, sumar Azure es el camino natural.

La pieza más característica es Synapse Analytics, que junta data warehouse, data lake y herramientas de integración en un solo producto. La idea es no tener que armarlo con piezas sueltas.

Componentes principales: **Azure Synapse Analytics, Azure Data Lake Storage, Azure Data Factory, Event Hubs, Azure Databricks y Power BI.**

3.4. Stack 4: Databricks Lakehouse

Databricks fue fundada por los mismos que crearon Apache Spark en Berkeley. Su aporte principal es la idea de *lakehouse*: combinar lo bueno de un data warehouse (consultas SQL rápidas, transacciones) con lo bueno de un data lake (almacenamiento barato, cualquier tipo de dato).

A diferencia de los stacks de los gigantes cloud, Databricks corre arriba de AWS, Azure o GCP. Es decir, no te casás con un solo proveedor. Donde más se usa es en empresas que mezclan analytics con machine learning.

Componentes principales: **Delta Lake, Apache Spark, Unity Catalog, MLflow y Databricks SQL.**

3.5. Stack 5: Snowflake

Snowflake apareció en 2014 con una propuesta distinta: hacer un data warehouse cloud desde cero, sin reciclar nada de los warehouses tradicionales como Teradata, Oracle o Vertica. Su idea más copiada fue separar almacenamiento de cómputo. El dato vive en object storage barato y aparte se levantan *warehouses* (clusters de cómputo) cuando hacen falta, y se apagan cuando no.

Otra cosa que ayudó a que explotara comercialmente es que corre sobre los tres grandes clouds (AWS, Azure, GCP) sin estar atado a ninguno. Su salida a bolsa en 2020 fue la IPO más grande de la historia del software.

Componentes principales: **Snowflake (motor), Snowpipe, Streams & Tasks, Snowpark, Snowflake Marketplace y Time Travel.**

3.6. Stack 6: Modern Data Stack (Best-of-Breed)

A diferencia de los anteriores, donde un solo proveedor te da todo, el llamado Modern Data Stack es más una filosofía: armar el stack con la mejor herramienta de cada categoría. Todas SaaS, todas conectándose entre sí. Empezó a tomar forma entre 2018 y 2020 con la maduración de productos como dbt, Fivetran y Looker.

Es el stack de moda en startups y empresas data-driven. Permite arrancar rápido porque todo viene plug-and-play y escala sin tener que reescribir la arquitectura. La contra es que la suma de varios contratos SaaS puede salir bastante cara.

Componentes típicos: **Fivetran** o **Airbyte** (ingesta), **dbt** (transformación), **Snowflake** o **BigQuery** (almacenamiento), **Airflow** o **Dagster** (orquestración), **Looker** o **Metabase** (visualización).

3.7. Stack 7: Confluent / Streaming

Todos los stacks anteriores están pensados principalmente para procesamiento por lotes (*batch*). El de Confluent va por otro lado: datos en tiempo real. El corazón es Apache Kafka, que nació en LinkedIn para mover billones de eventos por día entre sistemas.

Confluent es la empresa fundada por los creadores de Kafka, que vende una versión enterprise y managed. Este stack es lo que hay abajo de casi cualquier sistema moderno que tenga que reaccionar a eventos en milisegundos: detección de fraude, recomendaciones en vivo, tracking de logística, telemetría de IoT.

Componentes principales: **Apache Kafka**, **Kafka Connect**, **ksqlDB**, **Schema Registry**, **Apache Flink** y **Confluent Cloud**.

3.8. Stack 8: Hadoop Ecosystem (Open Source Tradicional)

Hadoop fue *el* stack de Big Data en la década del 2010. Salió de los papers de Google sobre MapReduce y GFS, y fue el primero que demostró que se podía procesar petabytes de datos arriba de hardware comercial. Yahoo!,

Facebook, Twitter y LinkedIn armaron sus primeras plataformas de datos sobre Hadoop.

Hoy ya se considera medio *legacy*. Mantener un cluster propio es un dolor de cabeza y los stacks cloud lo desplazaron en muchos casos. Igual sigue siendo importante: muchísimas empresas todavía lo usan en producción, y conceptualmente dio origen a casi todo lo que vino después (Spark, Hive, los formatos Parquet y ORC, etc.).

Componentes principales: **HDFS**, **Hive**, **Apache Spark**, **Apache Airflow**, **Presto/Trino** y **Apache Superset**.

4. Contenido de la Presentación

Cada grupo debe cubrir los siguientes puntos en su exposición. El orden es sugerido pero pueden adaptarlo.

4.1. Visión general del stack

- ¿Quién provee este stack? ¿Es de una empresa única o una combinación de herramientas independientes?
- ¿Cuándo surgió y qué problema vino a resolver?
- ¿Es propietario, open source, o híbrido?
- Modelo de negocio: ¿cómo se cobra? (pago por uso, suscripción, licencias, etc.)

4.2. Diagrama de arquitectura

- Hagan un diagrama que muestre cómo fluye un dato desde su origen hasta su consumo final.
- Identifiquen claramente las etapas: **ingesta**, **almacenamiento**, **procesamiento/transformación** y **consumo**.
- Indiquen qué componente del stack cubre cada etapa.

4.3. Componente por componente

Para cada herramienta principal del stack, expliquen:

- Qué hace y por qué existe.
- Qué tipo de datos maneja (estructurados, semi-estructurados, no estructurados).
- Cómo se integra con el resto del stack.

4.4. Conexión con los conceptos del curso

Esta sección es **obligatoria** y es la que más valoramos. Respondan:

- ¿Qué componentes del stack son **SQL/relacionales** y cuáles son **NoSQL**? Si hay NoSQL, ¿de qué tipo (documental, clave-valor, columnar, grafos)?
- ¿Dónde aparecen los **trade-offs del teorema CAP** en este stack? ¿Hay componentes que prioricen consistencia y otros disponibilidad?
- ¿Hay algún componente que use un **modelo de datos o paradigma que no vimos en clase**? Nómbrénlo (no hace falta que lo expliquen en profundidad).

4.5. Casos de uso reales

- Mencionen al menos **2 empresas conocidas** que usen este stack y para qué.
- ¿En qué tipo de industria o escenario es la mejor opción?
- ¿En qué casos **no** conviene usar este stack?

4.6. Comparación con otro stack

- Comparen su stack con al menos uno de los otros que están investigando sus compañeros.
- ¿Por qué una empresa elegiría este sobre el otro?

- ¿Qué tan difícil sería migrar de uno al otro? (concepto de *vendor lock-in*)

5. Criterios de Evaluación

Criterio	Qué evaluamos	Peso
Rigor técnico	Que la información sea correcta y no marketing. Que entiendan lo que están explicando.	30 %
Conexión con el curso	Calidad de la sección que relaciona el stack con los conceptos vistos (SQL/NoSQL, CAP, etc.).	25 %
Diagrama y claridad	Que el diagrama de arquitectura sea claro y que la presentación tenga buena estructura.	20 %
Casos de uso y comparación	Profundidad del análisis de cuándo conviene el stack y comparación con alternativas.	15 %
Respuesta a preguntas	Capacidad del grupo para responder preguntas del docente y de sus compañeros.	10 %

6. Recomendaciones

- **No se queden en el marketing.** Las páginas oficiales venden el producto. Busquen blogs técnicos, comparaciones independientes y casos de estudio reales. Los blogs de ingeniería de Spotify, Netflix, Uber o Airbnb suelen publicar sobre sus stacks.
- **No traten de cubrir todo.** Es mejor explicar 4-5 componentes bien que listar 15 superficialmente.
- **Practiquen la presentación** antes de la clase. 12-15 minutos pasan rápido.